Structured Query Language (SQL)

Essential Commands & Query Patterns

Database Systems Course

CS Department

March 17, 2026

Lecture Roadmap

1. Introduction
2. Basic Queries
3. Filtering Data
4. Aggregation
5. Joins
6. Subqueries

What is SQL?

Definition

Structured Query Language is the standard language for relational database management systems (RDBMS).

What is SQL?

Definition

Structured Query Language is the standard language for relational database management systems (RDBMS).

Key Categories

- **DDL** – Data Definition Language (CREATE, ALTER, DROP)
- **DML** – Data Manipulation Language (SELECT, INSERT, UPDATE)
- **DCL** – Data Control Language (GRANT, REVOKE)

What is SQL?

Definition

Structured Query Language is the standard language for relational database management systems (RDBMS).

Key Categories

- **DDL** – Data Definition Language (CREATE, ALTER, DROP)
- **DML** – Data Manipulation Language (SELECT, INSERT, UPDATE)
- **DCL** – Data Control Language (GRANT, REVOKE)

Today's Focus

We concentrate on **querying** and **manipulating** existing data using DML commands.

Sample Database Schema

Tables:

- `employees` (id, name, dept_id, salary, hire_date)
- `departments` (id, name, location)
- `projects` (id, title, budget)

Sample Database Schema

Tables:

- `employees` (id, name, dept_id, salary, hire_date)
- `departments` (id, name, location)
- `projects` (id, title, budget)

Relationships:

- Employees belong to Departments (Many-to-One)
- Employees work on Projects (Many-to-Many via `works_on`)

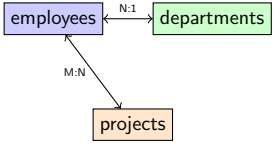
Sample Database Schema

Tables:

- `employees` (id, name, dept_id, salary, hire_date)
- `departments` (id, name, location)
- `projects` (id, title, budget)

Relationships:

- Employees belong to Departments (Many-to-One)
- Employees work on Projects (Many-to-Many via `works_on`)



The SELECT Statement

Syntax Structure

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

The SELECT Statement

Syntax Structure

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example 1: Specific Columns

```sql
SELECT name, salary
FROM employees
WHERE dept_id = 1;
```

The SELECT Statement

Syntax Structure

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example 1: Specific Columns

```sql
SELECT name, salary
FROM employees
WHERE dept_id = 1;
```

Returns: Names and salaries for employees in department 1.

The SELECT Statement

Syntax Structure

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example 1: Specific Columns

```sql
SELECT name, salary
FROM employees
WHERE dept_id = 1;
```

Returns: Names and salaries for employees in department 1.

Example 2: Wildcards

```sql
SELECT * FROM employees;
```

The SELECT Statement

Syntax Structure

```sql
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example 1: Specific Columns

```sql
SELECT name, salary
FROM employees
WHERE dept_id = 1;
```

Returns: Names and salaries for employees in department 1.

Example 2: Wildcards

```sql
SELECT * FROM employees;
```

Filtering with WHERE

Comparison Operators

- =, <> or !=, <, >, <=, >=
- `BETWEEN`, `LIKE`, `IN`
- `IS NULL`, `IS NOT NULL`

Filtering with WHERE

Comparison Operators

- =, <> or !=, <, >, <=, >=
- BETWEEN, LIKE, IN
- IS NULL, IS NOT NULL

Pattern Matching

```sql
-- Names starting with 'A'
WHERE name LIKE 'A%'

-- Single character wildcard
WHERE name LIKE '_ohn'
```

Filtering with WHERE

Comparison Operators

- =, <> or !=, <, >, <=, >=
- BETWEEN, LIKE, IN
- IS NULL, IS NOT NULL

Pattern Matching

```sql
-- Names starting with 'A'
WHERE name LIKE 'A%'

-- Single character wildcard
WHERE name LIKE '_ohn'
```

Logical Operators

```sql
WHERE (salary > 50000 AND dept_id = 2)
   OR hire_date > '2023-01-01'
```

Filtering with WHERE

Comparison Operators

- =, <> or !=, <, >, <=, >=
- BETWEEN, LIKE, IN
- IS NULL, IS NOT NULL

Pattern Matching

```sql
-- Names starting with 'A'
WHERE name LIKE 'A%'

-- Single character wildcard
WHERE name LIKE '_ohn'
```

Logical Operators

```sql
WHERE (salary > 50000 AND dept_id = 2)
   OR hire_date > '2023-01-01'
```

Use parentheses to control operator precedence!

Sorting and Pagination

ORDER BY

```sql
SELECT name, salary
FROM employees
WHERE salary > 40000
ORDER BY salary DESC, name ASC;
```

Sorting and Pagination

ORDER BY

```sql
SELECT name, salary
FROM employees
WHERE salary > 40000
ORDER BY salary DESC, name ASC;
```

- DESC: Descending
- ASC: Ascending (default)

Sorting and Pagination

ORDER BY

```sql
SELECT name, salary
FROM employees
WHERE salary > 40000
ORDER BY salary DESC, name ASC;
```

- DESC: Descending
- ASC: Ascending (default)

Limiting Results

Different SQL dialects:

```sql
-- MySQL/PostgreSQL
SELECT * FROM employees LIMIT 10 OFFSET 20;

-- SQL Server
SELECT TOP 10 * FROM employees;

-- Oracle (12c+)
```

Aggregate Functions

Common Functions

Function	Description
`COUNT(*)`	Count rows
`SUM(column)`	Total sum
`AVG(column)`	Average value
`MAX(column)/MIN(column)`	Extreme values

Aggregate Functions

Common Functions

Function	Description
COUNT(*)	Count rows
SUM(column)	Total sum
AVG(column)	Average value
MAX(column)/MIN(column)	Extreme values

Usage

```sql
SELECT
    COUNT(*) as total_employees,
    AVG(salary) as avg_salary,
    MAX(hire_date) as newest_hire
FROM employees;
```

Aggregate Functions

Common Functions

Function	Description
`COUNT(*)`	Count rows
`SUM(column)`	Total sum
`AVG(column)`	Average value
`MAX(column)/MIN(column)`	Extreme values

Usage

```sql
SELECT
    COUNT(*) as total_employees,
    AVG(salary) as avg_salary,
    MAX(hire_date) as newest_hire
FROM employees;
```

Note: Aggregate functions ignore `NULL` values except `COUNT(*)`.

Grouping Data

Concept

GROUP BY collapses rows with identical values into summary rows, typically used with aggregates.

Grouping Data

Concept

GROUP BY collapses rows with identical values into summary rows, typically used with aggregates.

Department Statistics

```sql
SELECT
    d.name as department,
    COUNT(e.id) as emp_count,
    AVG(e.salary) as avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.id
GROUP BY d.name;
```

Grouping Data

Concept

GROUP BY collapses rows with identical values into summary rows, typically used with aggregates.

Department Statistics

```sql
SELECT
    d.name as department,
    COUNT(e.id) as emp_count,
    AVG(e.salary) as avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.id
GROUP BY d.name;
```

The Golden Rule

Any column in `SELECT` that isn't an aggregate must appear in `GROUP BY` clause!

Filtering Groups with HAVING

WHERE vs HAVING

- **WHERE**: Filters *rows* before grouping
- **HAVING**: Filters *groups* after aggregation

Filtering Groups with HAVING

WHERE vs HAVING

- **WHERE**: Filters *rows* before grouping
- **HAVING**: Filters *groups* after aggregation

Execution Order

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Filtering Groups with HAVING

WHERE vs HAVING

- **WHERE**: Filters *rows* before grouping
- **HAVING**: Filters *groups* after aggregation

Execution Order

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Example

```sql
SELECT dept_id, AVG(salary) as avg_sal
FROM employees
WHERE hire_date > '2020-01-01'
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

Filtering Groups with HAVING

WHERE vs HAVING

- **WHERE**: Filters *rows* before grouping
- **HAVING**: Filters *groups* after aggregation

Execution Order

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Example

```sql
SELECT dept_id, AVG(salary) as avg_sal
FROM employees
WHERE hire_date > '2020-01-01'
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

Understanding JOINs

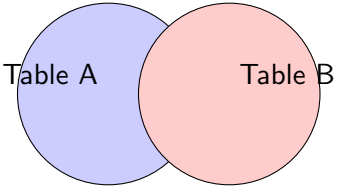
Definition

Combines rows from two or more tables based on a related column.

Understanding JOINs

Definition

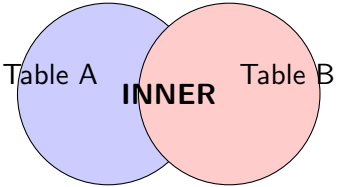
Combines rows from two or more tables based on a related column.



Understanding JOINs

Definition

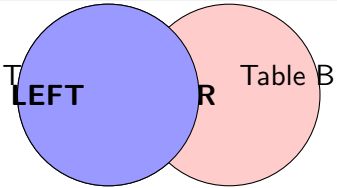
Combines rows from two or more tables based on a related column.



Understanding JOINs

Definition

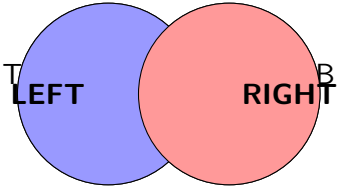
Combines rows from two or more tables based on a related column.



Understanding JOINs

Definition

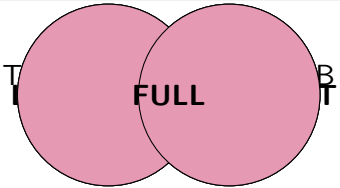
Combines rows from two or more tables based on a related column.



Understanding JOINs

Definition

Combines rows from two or more tables based on a related column.



JOIN Syntax

INNER JOIN (Intersection)

```sql
SELECT e.name, d.location
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

JOIN Syntax

INNER JOIN (Intersection)

```sql
SELECT e.name, d.location
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

Only employees with valid department IDs.

JOIN Syntax

INNER JOIN (Intersection)

```sql
SELECT e.name, d.location
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

Only employees with valid department IDs.

LEFT JOIN (Preserve Left Table)

```sql
SELECT d.name, COUNT(e.id) as emp_count
FROM departments d
LEFT JOIN employees e ON d.id = e.dept_id
GROUP BY d.name;
```

JOIN Syntax

INNER JOIN (Intersection)

```sql
SELECT e.name, d.location
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

Only employees with valid department IDs.

LEFT JOIN (Preserve Left Table)

```sql
SELECT d.name, COUNT(e.id) as emp_count
FROM departments d
LEFT JOIN employees e ON d.id = e.dept_id
GROUP BY d.name;
```

Shows all departments, even those with 0 employees.

Subqueries (Nested SELECTs)

Definition

A query nested inside another query. Can appear in `SELECT`, `FROM`, or `WHERE` clauses.

Subqueries (Nested SELECTs)

Definition

A query nested inside another query. Can appear in SELECT, FROM, or WHERE clauses.

Correlated

```sql
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Subqueries (Nested SELECTs)

Definition

A query nested inside another query. Can appear in SELECT, FROM, or WHERE clauses.

Correlated

```sql
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Executes once per row.

Subqueries (Nested SELECTs)

Definition

A query nested inside another query. Can appear in SELECT, FROM, or WHERE clauses.

Correlated

```sql
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Executes once per row.

Non-Correlated

```sql
SELECT name
FROM employees
WHERE dept_id IN (
    SELECT id
    FROM departments
    WHERE location = 'NYC'
);
```

Subqueries (Nested SELECTs)

Definition

A query nested inside another query. Can appear in SELECT, FROM, or WHERE clauses.

Correlated

```sql
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Executes once per row.

Non-Correlated

```sql
SELECT name
FROM employees
WHERE dept_id IN (
    SELECT id
    FROM departments
    WHERE location = 'NYC'
);
```

Executes once total.

Modifying Data

Safety First

Always use `WHERE` clauses with UPDATE and DELETE, or you'll affect **all** rows!

Modifying Data

Safety First

Always use WHERE clauses with UPDATE and DELETE, or you'll affect **all** rows!

INSERT

```sql
-- Single row
INSERT INTO employees (id, name, salary)
VALUES (101, 'Alice Smith', 75000);

-- Multiple rows
INSERT INTO employees VALUES
    (102, 'Bob', 2, 60000, '2024-01-15'),
    (103, 'Carol', 2, 65000, '2024-01-20');
```

Modifying Data

☞ Safety First

Always use `WHERE` clauses with UPDATE and DELETE, or you'll affect **all** rows!

INSERT

```sql
-- Single row
INSERT INTO employees (id, name, salary)
VALUES (101, 'Alice Smith', 75000);

-- Multiple rows
INSERT INTO employees VALUES
    (102, 'Bob', 2, 60000, '2024-01-15'),
    (103, 'Carol', 2, 65000, '2024-01-20');
```

UPDATE & DELETE

```sql
UPDATE employees
SET salary = salary * 1.05
```

Best Practices & Tips

Performance

- Use `EXPLAIN` to analyze queries
- Index columns in `WHERE`, `JOIN`, `ORDER BY`
- Avoid `SELECT *` in production
- Use `LIMIT` during development

Best Practices & Tips

Performance

- Use `EXPLAIN` to analyze queries
- Index columns in `WHERE`, `JOIN`, `ORDER BY`
- Avoid `SELECT *` in production
- Use `LIMIT` during development

Readability

- Use table aliases (`e`, `d`)
- Capitalize keywords
- Indent subqueries
- Comment complex logic

Best Practices & Tips

Performance

- Use `EXPLAIN` to analyze queries
- Index columns in `WHERE`, `JOIN`, `ORDER BY`
- Avoid `SELECT *` in production
- Use `LIMIT` during development

Readability

- Use table aliases (`e`, `d`)
- Capitalize keywords
- Indent subqueries
- Comment complex logic

Quick Reference

`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`

Questions?

Try These:

- Find employees earning above their department average
- List departments with no employees
- Update salaries based on tenure